

Legal AI Benchmark System: Implementation Foundation

Status: V0 implementation specification | 27 August 2026

The system compares a direct base model with a custom legal AI harness on fixed cases. The benchmark runner provides identical public inputs, keeps evaluation data private, and records quality, consistency, time, and cost. V0 supports text and JSON. The same foundation can later support tools, agents, several models, and document artifacts.

System components

Component	Responsibility	Main interface or output
Case store	Keeps versioned public tasks and private evaluation data.	PublicCase + PrivateCase
Dataset adapter	Maps LegalBench, CUAD, and custom data into one case format.	loadCase()
Model client	Calls Qwen, Ollama, or an API and records model usage.	generate()
System adapter	Runs Qwen direct or the custom harness through one contract.	run(case) -> RunResult
Runner	Selects cases and systems, enforces limits, and records every run.	RunContext + run ID
Graders	Compare the result with private exact, span, or rubric criteria.	grade() -> GradeResult
Results and report	Save answers, traces, scores, errors, time, tokens, and cost.	Per-suite report

Interaction

The runner loads one case. The dataset adapter exposes only its public task and inputs. The runner calls each system through the same interface. The direct system makes one strong Qwen call. The harness can use several calls, agents, tools, or models. Each system returns a normalized RunResult. The grader then receives the result and private evaluation data. The reporter compares systems by benchmark suite.

```
SystemUnderTest.run(PublicCase, RunContext) -> RunResult
```

```
Grader.grade(PublicCase, PrivateCase, RunResult) -> GradeResult
```

Initial benchmark scope

LegalBench: 15-20 short reasoning cases; exact-match or classification grading.

CUAD: 10-15 contract clause cases; span precision and recall.

Custom or LAB-derived: 3-5 legal workflow cases; atomic rubric criteria and strict all-pass grading.

Report each suite separately. Also report consistency, failures, model calls, tokens, latency, and cost.

Controls, assumptions, and open question

Controls: Use fixed cases, output schemas, graders, limits, model IDs, and locked test data. Tune the harness only on development cases.

Assumption: V0 accepts text and JSON. Add DOCX and XLSX artifacts after the runner and graders work.

Open question: Select the first custom legal workflow and the person who will approve its rubric.

First implementation

Define the shared types. Then build the case loader, result store, model client, Qwen-direct adapter, harness adapter, three graders, and Markdown reporter. Use one repetition while you build the plumbing. Use at least three repetitions for the first comparison.